Test Data Set

IBA Campus Facility Booking System

Team Members:
Abdul Wasay Imran (27126), Saad Imam (27079),
Muhammad Imad Raza (26953), Syed Muhammad Deebaj Kazmi (26012)

Version 1.1 — May 28, 2026

Contents

1 Overview **2**

2 Environment Configuration **2**

3 Test Data Summary **2**
3.1 Test User Accounts 2
3.2 Facility Data (Buildings & Rooms) 2

4 Seeding & Reset Logic **3**

A Appendix: Source Scripts **4**
A.1 init.sql (Full Schema Setup) 4
A.2 seedTestData.sql (Data Injection) 9
A.3 seed.helper.ts (TypeScript Reset Helper) 11

1 Overview

This document defines the data environment used for the Quality Assurance lifecycle of the IBA Campus Facility Booking System. To ensure test repeatability and prevent ”flaky” tests, the system utilizes a State Reset Strategy.

The environment is completely wiped and re-seeded before every E2E test suite and during specific integration tests. This ensures that every test starts from a known, clean state.

2 Environment Configuration

The testing environment is isolated from production using a dedicated Supabase PostgreSQL instance.

- **Database Engine:** PostgreSQL (via Supabase)
- **Reset Method:** Drop and Recreate Schema (`init.sql`)
- **Seeding Method:** Static UUID Injection (`seedTestData.sql`)
- **Automation Helper:** TypeScript / Node.js `pg` client

3 Test Data Summary

The following tables represent the static data injected into the system during the seeding process. These IDs are hardcoded in our SQL scripts to allow E2E tests (Playwright/Selenium) to target specific entities consistently.

3.1 Test User Accounts

Note: All test accounts use the password: ***testpass***

ERP / Username	Name	Role	Email
test-student	Test Student	Student	student@test.com
test-po	Test PO	Program Office	po@test.com
test-admin	Test Admin	Admin	admin@test.com
test-student-2	Concurrency Student	Student	student2@test.com

3.2 Facility Data (Buildings & Rooms)

Static UUIDs are used to ensure API calls in integration tests remain valid across environment resets.

Entity Name	Type	Static UUID Reference
Test Building	Building	`99999999-9999-4999-8999-999999999999`
Test Room A	Room	`aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa`
Test Room B	Room	`bbbbbbbb-bbbb-4bbb-8bbb-bbbbbbbbbbbb`

4 Seeding & Reset Logic

The system uses a two-step process to maintain data integrity during testing:

1. **Global Setup:** The `global-setup.ts` script triggers a database reset before the test runner (Playwright) starts.
2. **Atomic Reset:** The `resetDatabase()` function executes the SQL scripts via a PostgreSQL client, ensuring the `bookings` table is cleared and the `time_slots` are restored.

A Appendix: Source Scripts

A.1 init.sql (Full Schema Setup)

This script is used to initialize the database from scratch, creating all tables, enums, and static time slots.

```
-- Full Reset: Drop everything first
DROP TABLE IF EXISTS bookings cascade;


DROP TABLE IF EXISTS blocked_slots cascade;


DROP TABLE IF EXISTS rooms cascade;


DROP TABLE IF EXISTS buildings cascade;


DROP TABLE IF EXISTS users cascade;


DROP TABLE IF EXISTS time_slots cascade;


DROP TYPE if EXISTS user_role cascade;


DROP TYPE if EXISTS booking_status cascade;


DROP TYPE if EXISTS room_type cascade;


-- ============================================================
--  IBA FACILITY BOOKING SYSTEM     Supabase SQL Schema
--  Run this entire file in: Supabase     SQL Editor     New Query
-- ============================================================
--        ENUMS


CREATE TYPE user_role AS ENUM('student', 'programoffice', 'admin');


CREATE TYPE booking_status AS ENUM('pending', 'approved', 'rejected', '
    cancelled');


CREATE TYPE room_type AS ENUM(
    'Classroom',
    'Seminar Hall',
    'Computer Lab',
    'Meeting Room'
);


--        USERS


CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT GEN_RANDOM_UUID(),
    erp VARCHAR(50) UNIQUE NOT NULL, -- ERP ID for students, username for
```

```
    others
    name VARCHAR(150) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL, -- bcrypt hashed
    role user_role NOT NULL DEFAULT 'student',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);


--        BUILDINGS


CREATE TABLE buildings (
    id UUID PRIMARY KEY DEFAULT GEN_RANDOM_UUID(),
    name VARCHAR(150) NOT NULL,
    location VARCHAR(200),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);


--        ROOMS


CREATE TABLE rooms (
    id UUID PRIMARY KEY DEFAULT GEN_RANDOM_UUID(),
    building_id UUID NOT NULL REFERENCES buildings (id) ON DELETE CASCADE,
    name VARCHAR(100) NOT NULL,
    capacity INTEGER NOT NULL CHECK (capacity > 0),
    type room_type NOT NULL DEFAULT 'Classroom',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);


--        TIME SLOTS (static reference table)

CREATE TABLE time_slots (
    id INTEGER PRIMARY KEY,
    start_time VARCHAR(5) NOT NULL, -- e.g. "08:30"
    end_time VARCHAR(5) NOT NULL, -- e.g. "09:45"
    label VARCHAR(30) NOT NULL -- e.g. "8:30     9:45"
);


INSERT INTO
    time_slots (id, start_time, end_time, label)
VALUES
    (1, '08:30', '09:45', '8:30     9:45'),
    (2, '10:00', '11:15', '10:00     11:15'),
    (3, '11:30', '12:45', '11:30     12:45'),
    (4, '13:00', '14:15', '1:00     2:15'),
    (5, '14:30', '15:45', '2:30     3:45'),
    (6, '16:00', '17:15', '4:00     5:15'),
    (7, '17:30', '18:45', '5:30     6:45');


--        BOOKINGS


CREATE TABLE bookings (
    id UUID PRIMARY KEY DEFAULT GEN_RANDOM_UUID(),
    user_id UUID NOT NULL REFERENCES users (id) ON DELETE CASCADE,
    room_id UUID NOT NULL REFERENCES rooms (id) ON DELETE CASCADE,
```

```
    slot_id INTEGER NOT NULL REFERENCES time_slots (id),
    date date NOT NULL,
    purpose TEXT NOT NULL,
    status booking_status NOT NULL DEFAULT 'pending',
    reviewed_by UUID REFERENCES users (id), -- PO or admin who actioned it
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    -- Prevent double-booking same room+date+slot
    UNIQUE (room_id, date, slot_id)
);


--        BLOCKED SLOTS


CREATE TABLE blocked_slots (
    id UUID PRIMARY KEY DEFAULT GEN_RANDOM_UUID(),
    room_id UUID NOT NULL REFERENCES rooms (id) ON DELETE CASCADE,
    slot_id INTEGER NOT NULL REFERENCES time_slots (id),
    date date NOT NULL,
    reason VARCHAR(200),
    blocked_by UUID NOT NULL REFERENCES users (id),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (room_id, date, slot_id)
);


--        UPDATED_AT TRIGGER


CREATE OR REPLACE FUNCTION set_updated_at () returns trigger AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ language plpgsql;


CREATE TRIGGER trg_users_updated_at before
UPDATE ON users FOR each ROW
EXECUTE function set_updated_at ();


CREATE TRIGGER trg_bookings_updated_at before
UPDATE ON bookings FOR each ROW
EXECUTE function set_updated_at ();


--        INDEXES


CREATE INDEX idx_bookings_user_id ON bookings (user_id);


CREATE INDEX idx_bookings_room_id ON bookings (room_id);


CREATE INDEX idx_bookings_date ON bookings (date);


CREATE INDEX idx_bookings_status ON bookings (status);


```

```sql
CREATE INDEX idx_blocked_room_date ON blocked_slots (room_id, date);


CREATE INDEX idx_rooms_building ON rooms (building_id);


--         SEED DATA


-- Admin user (password: admin123)
INSERT INTO
    users (erp, name, email, password, role)
VALUES
    (
        'admin',
        'System Admin',
        'admin@iba.edu.pk',
        '$2a$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi',
        'admin'
    );


-- Program Office (password: password)
INSERT INTO
    users (erp, name, email, password, role)
VALUES
    (
        'po001',
        'Dr. Fatima Zahra',
        'fatima@iba.edu.pk',
        '$2a$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi',
        'programoffice'
    );


-- Students (password: password)
INSERT INTO
    users (erp, name, email, password, role)
VALUES
    (
        '12345',
        'Ali Hassan',
        'ali@iba.edu.pk',
        '$2a$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi',
        'student'
    ),
    (
        '22222',
        'Sara Khan',
        'sara@iba.edu.pk',
        '$2a$10$92IXUNpkjO0rOQ5byMi.Ye4oKoEa3Ro9llC/.og/at2.uheWG/igi',
        'student'
    );


-- Buildings
INSERT INTO
    buildings (id, name, location)
VALUES
    (
        '11111111-1111-1111-1111-111111111111',
        'Main Academic Block',
        'North Campus'
```

```
    ),
    (
        '22222222-2222-2222-2222-222222222222',
        'Business School',
        'East Wing'
    ),
    (
        '33333333-3333-3333-3333-333333333333',
        'Technology Block',
        'West Campus'
    );


-- Rooms
INSERT INTO
    rooms (building_id, name, capacity, type)
VALUES
    (
        '11111111-1111-1111-1111-111111111111',
        'Room 101',
        40,
        'Classroom'
    ),
    (
        '11111111-1111-1111-1111-111111111111',
        'Room 102',
        35,
        'Classroom'
    ),
    (
        '11111111-1111-1111-1111-111111111111',
        'Seminar Hall A',
        80,
        'Seminar Hall'
    ),
    (
        '11111111-1111-1111-1111-111111111111',
        'Computer Lab 1',
        30,
        'Computer Lab'
    ),
    (
        '22222222-2222-2222-2222-222222222222',
        'BS Conference Room',
        20,
        'Meeting Room'
    ),
    (
        '22222222-2222-2222-2222-222222222222',
        'Lecture Hall B1',
        60,
        'Classroom'
    ),
    (
        '33333333-3333-3333-3333-333333333333',
        'Tech Lab 01',
        25,
        'Computer Lab'
    ),
    (
        '33333333-3333-3333-3333-333333333333',
        'Innovation Hub',
        15,
```

```
        'Meeting Room'
    );


--        ROW LEVEL SECURITY (RLS)     disable for service role

-- We use the service role key in the backend so RLS won't block us.
-- But enable it on all tables for safety:
ALTER TABLE users enable ROW level security;


ALTER TABLE buildings enable ROW level security;


ALTER TABLE rooms enable ROW level security;


ALTER TABLE bookings enable ROW level security;


ALTER TABLE blocked_slots enable ROW level security;


ALTER TABLE time_slots enable ROW level security;


-- Allow service role to bypass RLS (this is the default in Supabase)
-- No additional policies needed since we use service_role key server-side.
```

A.2 seedTestData.sql (Data Injection)

This script truncates existing data and injects the specific records required for the test cases defined in the Test Plan.

```
-- 1. DELETE all existing data (Order matters due to Foreign Keys)
-- Delete from child tables first, then parent tables
DELETE FROM bookings;


DELETE FROM blocked_slots;


DELETE FROM rooms;


DELETE FROM buildings;


DELETE FROM users;


-- 2. INSERT exactly these users
-- Password hash for 'testpass' using bcrypt (standard cost 10)
-- $2a$10$4r8D5gJGiqNuaPTR6qrxHeYlkUKl2nba97oZH.55EMivPEW8HoLe2
INSERT INTO
    users (erp, name, email, password, role)
VALUES
    (
        'test-student',
        'Test Student',
        'student@test.com',
        '$2a$10$4r8D5gJGiqNuaPTR6qrxHeYlkUKl2nba97oZH.55EMivPEW8HoLe2',
```

```
        'student'
    ),
    (
        'test-po',
        'Test PO',
        'po@test.com',
        '$2a$10$4r8D5gJGiqNuaPTR6qrxHeYlkUKl2nba97oZH.55EMivPEW8HoLe2',
        'programoffice'
    ),
    (
        'test-admin',
        'Test Admin',
        'admin@test.com',
        '$2a$10$4r8D5gJGiqNuaPTR6qrxHeYlkUKl2nba97oZH.55EMivPEW8HoLe2',
        'admin'
    ),
    (
        'test-student-2',
        'Concurrency Student',
        'student2@test.com',
        '$2a$10$4r8D5gJGiqNuaPTR6qrxHeYlkUKl2nba97oZH.55EMivPEW8HoLe2',
        'student'
    );


-- 3. INSERT one building
INSERT INTO
    buildings (id, name, location)
VALUES
    (
        '99999999-9999-4999-8999-999999999999',
        'Test Building',
        'Main Campus'
    );


-- 4. INSERT two rooms
INSERT INTO
    rooms (id, building_id, name, capacity, type)
VALUES
    (
        'aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa',
        '99999999-9999-4999-8999-999999999999',
        'Test Room A',
        30,
        'Classroom'
    ),
    (
        'bbbbbbbb-bbbb-4bbb-8bbb-bbbbbbbbbbbb',
        '99999999-9999-4999-8999-999999999999',
        'Test Room B',
        20,
        'Meeting Room'
    );


-- 5. INSERT one existing booking for Test Room A, slot 1, a future date
-- We use current_date + interval '7 days' to ensure it's always in the future
INSERT INTO
    bookings (user_id, room_id, slot_id, date, purpose, status)
SELECT
    id,
    'aaaaaaaa-aaaa-4aaa-8aaa-aaaaaaaaaaaa',
```

```
    1,
    current_date + INTERVAL '7 days',
    'Existing Booking for Conflict Test',
    'approved'
FROM
    users
WHERE
    erp = 'test-student'
LIMIT
    1;
```

A.3 seed.helper.ts (TypeScript Reset Helper)

This function is called by the test runner to execute the reset logic programmatically.

```
import { Client } from "pg";
import * as fs from "fs";
import * as path from "path";
import * as dotenv from "dotenv";

// Load the environment variables from .env.test
dotenv.config({ path: ".env.test" });

export async function resetDatabase() {
    // Use DATABASE_URL from .env.test
    const connectionString = process.env.DATABASE_URL;

    if (!connectionString) {
        console.error("Error: DATABASE_URL not found in .env.test");
        process.exit(1);
    }

    const client = new Client({
        connectionString,
        ssl: { rejectUnauthorized: false }, // Required for Supabase remote connections
    });

    try {
        await client.connect();
        console.log(`Connected to Supabase.`);

        const filePath = path.join(__dirname, "./../../test-data", "seedTestData.sql");

        console.log(`Reading file: ${filePath}`);
        const sql = fs.readFileSync(filePath, "utf8");

        console.log(`Executing SQL...`);
        await client.query(sql);

        console.log(`    Successfully executed seedTestData.sql`);
    } catch (err: any) {
        console.error(`    Error during seeding:`);
        console.error(err.message);
        process.exit(1);
    } finally {
        await client.end();
    }
}
```